TalentFlow Sprint 2 Progress Update and Retrospective

Completion

The system is **25% complete** (46 of 187 development hours). Our timeline planned **43%** by Oct 24, so we are about 35 hours behind. Job postings (4.2.1) and resume upload (4.2.3.2) are done. The careers page and application form have not started, and the AI Engine (4.3), planned for Oct 18–24, has not started either.

Development work	Estimated (hrs)	Done (hrs)	Status
3.2.3.1 Hosting, Git & CI/CD	6	6	Done in Sprint 1
4.1 Authentication & Security	25	25	Done in Sprint 1
4.2.1 Job posting create, edit & open/close	10	10	Done in Sprint 2, merged (PR #21)
4.2.2 Public careers page	6	0	Moved to Sprint 3
4.2.3 Application form & resume upload	9	5	Resume upload done in Sprint 2, merged (PR #25); application form moved to Sprint 3
4.3 onward: remaining core modules	71	0	Not started
5. Testing & Quality Assurance	24	0	Not started
6. Deployment & Project Closure	37	0	Not started
Total	**187**	**46**	**25%**

Same method as Sprint 1: system development only, using the PERT expected hours from Project Timeline v2. Resume upload (4.2.3.2) counts its own 5 hours. Sprint 2 finished 15 of the 24 hours planned for it (63%). The planned 43% counts 4.2 (due Oct 17) and 4.3 (due Oct 24).

Completed in Sprint 2

Sprint 2 delivered job postings (4.2.1), resume upload (4.2.3.2), and the database and shared pieces for the rest of the portal.

WBS	Task
4.2.1	Job postings API: create, list, view, edit, open/close. Only managers can change postings; visitors only see open ones.
4.2.1.1	Job posting create form with a requirements editor and live preview
4.2.1.2	Edit page and open/close switch (closing asks to confirm); prompt before leaving unsaved changes

4.2.1	Backend unit and e2e tests for every job postings endpoint; frontend tests for the list, form, and requirements editor
4.2.3	Applications, applicant profiles, and resumes tables (migration 0002); resume upload token; shared careers service, public page layout, and resume picker
4.2.3.2	Resume upload API: PDF or Word up to 5 MB, checked by file contents and stored in the database; staff-only download
4.2.3.2	Resume upload unit and e2e tests, and the resume upload test matrix
—	Frontend code brought in line with the README Angular coding guidelines; Prettier set to 80 columns
—	Sprint 2 task document: each member's task, setup steps, and merge plan
—	Code review and approval of PR #21, #22, #23, and #25 before merging

Sprint 1 was released from dev to main on Oct 20 (PR #20). Job postings (PR #21), the database and shared pieces (PR #22), and resume upload (PR #25) were merged into dev on Oct 24.

Carried over

WBS	Task	Est. (hrs)	Moves to
4.2.2.1	Public careers page with open listings (responsive)	6	Sprint 3
4.2.3.1	Application form with validation	5	Sprint 3
—	Test matrix: job postings, careers page, application form	—	Sprint 3
3.3	AI proof of concept (prompt design, token and response-time checks)	16	Sprint 3, before the Gemini integration

Sprint 3 plan (Oct 25–31)

Sprint 3 first finishes the portal, then starts the AI Engine (4.3). The portal tasks were assigned on Oct 23 with a step-by-step task document, and each member got their own task document with the code on Oct 24. Each task can be done on its own now that PR #21, #22, and #25 are merged.

WBS	Task
4.2.2.1	Public careers page and role page (responsive)
4.2.3.1	Application API and apply page with validation
—	Backend tests for the application API
—	Frontend tests for the shared pieces and the careers and apply pages; careers and apply test matrix
3.3, 4.3	AI proof of concept, then Gemini prompt and API integration, resume text extraction, fit score UI
—	Sprint 3 progress document and Notion board updates

Each task is one branch and one pull request into dev.

Retrospective

What went well

- Job postings work end to end: managers create, edit, and open or close postings, and the API, pages, and tests are merged.
- Resume upload is done with unit and e2e tests and a test matrix, built on the shared database pieces without waiting on other tasks.
- The database for applications and resumes is designed and merged, so the remaining portal tasks no longer wait on it. Resumes are stored in the database because the API host deletes uploaded files on every deploy.
- Sprint 3 portal tasks are split so each member's branch builds and passes its tests on its own, and the branches merge in any order. This fixes Sprint 1's merge conflicts.
- The frontend now follows the README Angular coding guidelines, and Prettier keeps lines within 80 columns.

What could be improved

- We are further behind: about 35 hours, up from 24. The careers page and application form slipped again, and the AI Engine (4.3) has not started.
- Portal tasks were assigned late in the sprint (Oct 23), even though Sprint 1's action item was to assign them on day one. Sprint 3 tasks will be assigned on Oct 25, with task documents ready.
- Early code did not follow the README coding guidelines (interface instead of type, async/await, 100-column lines) and had to be reworked. Check new code against the guidelines before opening a pull request.
- Sprint 1 was released to main on Oct 20, three days after the planned Oct 17.

What needs to be done

- ☒ Release Sprint 1 from dev to main (Oct 20)
- ☒ Assign Sprint 2 portal tasks with a task document (Oct 23)
- ☒ Review and merge PR #21 (job postings) and PR #22 (database and shared pieces) into dev (Oct 24)
- ☒ Review and merge PR #25 (resume upload) into dev (Oct 24)
- ☐ Every member: pull dev, run the database migration on their own Neon branch, and start their task
- ☐ Finish the careers page and application form, with tests and test matrices
- ☐ Decide how job postings are deleted: the app closes postings instead, because applications refer to them, but the project requires Delete in its CRUD
- ☐ Bring the Sprint 1 frontend code (login, dashboard, team, auth service) in line with the coding guidelines
- ☐ Assign the AI Engine (4.3) tasks on Oct 25 and start the AI proof of concept (3.3)